



**EDUCACIÓN**  
SECRETARÍA DE EDUCACIÓN PÚBLICA



# **TECNOLÓGICO NACIONAL DE MÉXICO INSTITUTO TECNOLÓGICO DE TIJUANA**

**SUBDIRECCIÓN ACADÉMICA  
DEPARTAMENTO DE SISTEMAS Y COMPUTACIÓN**

**SEMESTRE:**

Agosto – Diciembre 2025

**CARRERA:**

Ingeniería Informática

**MATERIA:**

Patrones de diseño

**TÍTULO ACTIVIDAD:**

Examen Unidad 2

**UNIDAD A EVALUAR:**

Unidad 2

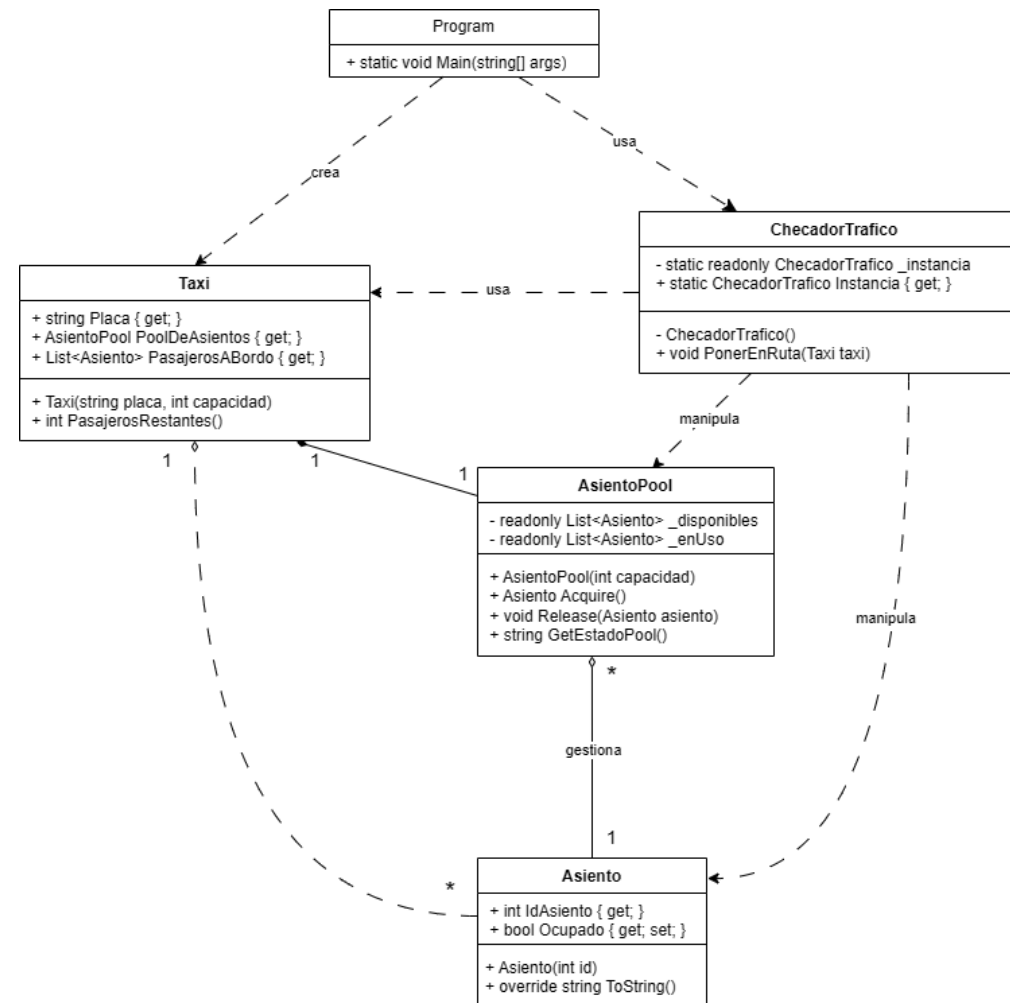
**NOMBRE Y NÚMERO DE CONTROL DEL ALUMNO:**

Angulo Berrelleza Fernando 21212322

**NOMBRE DEL MAESTRO (A):**

Maribel Guerrero Luis

## Diagrama UML



## Program

```
namespace ExamenU2
{
    0 referencias
    internal class Program
    {
        0 referencias
        static void Main(string[] args)
        {
            Console.WriteLine("VALIDANDO SINGLETON...");
            ChecadorTrafico checador1 = ChecadorTrafico.Instancia;
            ChecadorTrafico checador2 = ChecadorTrafico.Instancia;

            Console.WriteLine(ReferenceEquals(checador1, checador2));

            Taxi miTaxi = new Taxi("T-123-XYZ");
            Console.ReadKey();
            Console.Clear();

            checador1.PonerEnRuta(miTaxi);

            Console.ReadKey();
        }
    }
}
```

## Asiento.cs

```
namespace ExamenU2
{
    14 referencias
    public class Asiento
    {
        4 referencias
        public int IdAsiento { get; private set; }
        4 referencias
        public bool Ocupado { get; set; }

        1 referencia
        public Asiento(int id)
        {
            this.IdAsiento = id;
            this.Ocupado = false;
        }

        0 referencias
        public override string ToString()
        {
            string estado = Ocupado ? "Ocupado" : "Libre";
            return $"Asiento No. {IdAsiento} ({estado})";
        }
    }
}
```

## AsientoPool.cs

3 referencias

```
public class AsientoPool
```

```
{
```

```
    private readonly List<Asiento> _disponibles;  
    private readonly List<Asiento> _enUso;
```

1 referencia

```
    public AsientoPool(int capacidad)
```

```
    {
```

```
        _disponibles = new List<Asiento>();  
        _enUso = new List<Asiento>();
```

```
        for (int i = 0; i < capacidad; i++)  
        {  
            _disponibles.Add(new Asiento(i + 1));  
        }
```

```
        Console.WriteLine($"POOL: Creando una piscina con {capacidad} asientos.");
```

```
    }
```

1 referencia

```
    public Asiento Acquire()
```

```
    {
```

```
        if (_disponibles.Count == 0)  
        {  
            Console.WriteLine("POOL: ¡Advertencia! No hay asientos disponibles en la piscina.");  
            return null;  
        }
```

```
        Asiento asiento = _disponibles[0];  
        _disponibles.RemoveAt(0);
```

```
        asiento.Ocupado = true;  
        _enUso.Add(asiento);
```

```
        Console.WriteLine($"POOL: Asiento {asiento.IdAsiento} adquirido. Disponibles: {_disponibles.Count}");  
        return asiento;
```

```
    }
```

2 referencias

```
    public void Release(Asiento asiento)
```

```
    {
```

```
        if (asiento != null && _enUso.Contains(asiento))  
        {  
            asiento.Ocupado = false;  
            _enUso.Remove(asiento);  
            _disponibles.Add(asiento);  
            Console.WriteLine($"POOL: Asiento {asiento.IdAsiento} Desocupado. Disponibles: {_disponibles.Count}");  
        }
```

```
    }
```

3 referencias

```
    public string GetEstadoPool()
```

```
    {
```

```
        return $"Disponibles: {_disponibles.Count}, En Uso: {_enUso.Count}";
```

```
    }
```

```
}
```

## ChecadorTrafico.cs

```
8 referencias
public sealed class ChecadorTrafico
{
    private static readonly ChecadorTrafico _instancia = new ChecadorTrafico();
    1 referencia
    private ChecadorTrafico()
    {
        Console.WriteLine($"n--- SINGLETON: Checador de Tráfico '{this.GetHashCode()}' inicializado. ---");
    }

    // 3. La "Instancia" tiene la lógica de bloqueo
    2 referencias
    public static ChecadorTrafico Instancia
    {
        get { return _instancia; }
    }

    1 referencia
    public void PonerEnRuta(Taxi taxi)
    {
        // DESTINO 1: Llenar el taxi
        Console.WriteLine($"n\tINICIO DE RUTA");
        Console.WriteLine($"TAXI {taxi.Placa}: subiendo pasajeros");
        for (int i = 0; i < 15; i++)
        {
            Asiento asiento = taxi.PoolDeAsientos.Acquire();

            if (asiento != null)
            {
                taxi.PasajerosABordo.Add(asiento);
            }

            Thread.Sleep(200);
        }

        // comprueba el estado del pool DIRECTAMENTE
        Console.WriteLine($"nTAXI LLENO. Estado del Pool: {taxi.PoolDeAsientos.GetEstadoPool()}");
        Console.ReadKey();

        // DESTINO 2: Parada intermedia
        Console.WriteLine($"n\tLLEGADA A DESTINO 2");
        int cantidadBajar = 5;
        Console.WriteLine($"TAXI {taxi.Placa}: Van a bajar {cantidadBajar} pasajeros");
    }
}
```

```
for (int i = 0; i < cantidadBajar; i++)
{
    if (taxi.PasajerosABordo.Any())
    {
        Asiento asientoALiberar = taxi.PasajerosABordo[0];
        taxi.PasajerosABordo.RemoveAt(0);

        taxi.PoolDeAsientos.Release(asientoALiberar);
    }
}

Console.WriteLine($"nPARADA INTERMEDIA. Estado del Pool: {taxi.PoolDeAsientos.GetEstadoPool()}");
Console.ReadKey();

// DESTINO 3: Fin de la ruta
Console.WriteLine($"n\tLLEGADA A DESTINO FINAL 3");
int restantes = taxi.PasajerosRestantes();
Console.WriteLine($"TAXI {taxi.Placa}: Van a bajar {restantes} pasajeros");

// Usamos un bucle while para vaciar el taxi
while (taxi.PasajerosABordo.Any())
{
    Asiento asientoALiberar = taxi.PasajerosABordo[0];
    taxi.PasajerosABordo.RemoveAt(0);

    taxi.PoolDeAsientos.Release(asientoALiberar);
}

Console.WriteLine($"nFIN DE RUTA. Estado del Pool: {taxi.PoolDeAsientos.GetEstadoPool()}");
Console.WriteLine("Todos los asientos han sido devueltos a la piscina y están listos.");
}
```

## Taxi.cs

4 referencias

```
public class Taxi
```

```
{
```

4 referencias

```
public string Placa { get; private set; }
```

7 referencias

```
public AsientoPool PoolDeAsientos { get; private set; }
```

9 referencias

```
public List<Asiento> PasajerosABordo { get; private set; }
```

1 referencia

```
public Taxi(string placa, int capacidad = 15)
```

```
{
```

```
    this.Placa = placa;
```

```
    this.PoolDeAsientos = new AsientoPool(capacidad);
```

```
    this.PasajerosABordo = new List<Asiento>();
```

```
}
```

1 referencia

```
public int PasajerosRestantes() => PasajerosABordo.Count;
```

```
}
```

## Conclusión

En este proyecto, usamos dos patrones de diseño para hacer un simulador de taxis: el Singleton y el Object Pool. El Singleton lo usamos para el ChecadorTrafico, que es como el jefe de la estación. Nos aseguramos de que solo hubiera uno solo de estos en todo el programa, y él era el que mandaba al taxi a su ruta. Para el Object Pool, lo usamos para los asientos. En vez de crear asientos nuevos y borrarlos cada vez, hicimos una "piscina" con los 15 asientos del taxi para poder reutilizarlos.